Atividade de Pesquisa e Arquitetura: Desafio SQL vs. NoSQL

A escolha de um banco de dados depende dos requisitos do sistema, como consistência, disponibilidade, volume de dados, estrutura das informações, padrão de acesso e necessidade de escala. A seguir são apresentados os principais conceitos usados para analisar os cenários.

1. FUNDAMENTOS TEÓRICOS

1.1 ACID e BASE

ACID representa quatro propriedades de transações: Atomicidade, Consistência, Isolamento e Durabilidade. Elas garantem que uma transação seja executada de forma completa, mantenha as regras de integridade, controle operações concorrentes e preserve as alterações confirmadas mesmo após falhas [1].

BASE significa Basically Available, Soft State e Eventual Consistency. É uma abordagem comum em sistemas distribuídos que prioriza disponibilidade e escalabilidade, aceitando que os dados entre diferentes réplicas possam ficar temporariamente inconsistentes até convergirem [2].

1.2 Teorema CAP

O Teorema CAP trata de três propriedades de sistemas distribuídos: Consistência, Disponibilidade e Tolerância a Partições. Quando ocorre uma partição de rede, não é possível garantir ao mesmo tempo consistência forte e disponibilidade total. Por isso, a arquitetura deve definir qual comportamento é mais importante durante uma falha de comunicação [3].

1.3 Schema-on-write e Schema-on-read

Schema-on-write define e valida a estrutura dos dados antes da gravação. É comum em bancos relacionais, nos quais tabelas, tipos e relacionamentos são definidos previamente. Schema-on-read permite maior flexibilidade, deixando parte da interpretação da estrutura para o momento da leitura. Essa característica é útil quando os dados possuem formatos variados [4].

1.4 Escalabilidade

Scale-up, ou escalabilidade vertical, aumenta os recursos de um único servidor, como processador, memória e armazenamento. Scale-out, ou escalabilidade horizontal, adiciona novos servidores ao sistema. O sharding é uma técnica de scale-out que divide os dados entre diferentes servidores para distribuir a carga [5].

1.5 Famílias de bancos NoSQL

Documentos: armazenam informações em documentos, normalmente em formatos como JSON ou BSON. Permitem estruturas flexíveis e são adequados para catálogos, perfis de usuários e aplicações em que os registros podem possuir atributos diferentes. Um exemplo é o MongoDB [6].

Chave-valor: associa uma chave a um valor e é especialmente eficiente quando a aplicação precisa encontrar rapidamente um dado a partir de sua chave. É comum em cache, sessões, contadores e carrinhos de compras. Um exemplo é o Redis [7].

Colunar (wide-column): organiza os dados em partições e colunas, sendo indicado para grandes volumes distribuídos e cargas intensivas de escrita e leitura. É utilizado em cenários como IoT, telemetria e grandes quantidades de eventos. Um exemplo é o Apache Cassandra [8].

Grafos: representam entidades como nós e os relacionamentos entre elas como conexões. São indicados quando os relacionamentos são a parte principal das consultas, como em redes sociais, sistemas de recomendação e detecção de fraudes. Um exemplo é o Neo4j [9].

2. ANÁLISE DOS CENÁRIOS

2.1 Cenário A — Core Bancário e Processamento de PIX

O sistema precisa garantir que o saldo de uma conta não fique inconsistente, que débito e crédito façam parte da mesma operação e que exista histórico completo das transações.

Item de Avaliação	Descrição dos Requisitos
Paradigma Escolhido	SQL (Relacional). O modelo relacional é adequado para dados estruturados, relacionamentos e regras de integridade.
SGBD Recomendado	PostgreSQL. Possui suporte a transações, controle de concorrência, restrições de integridade e propriedades ACID [1].
Justificativa Técnica	A prioridade é a consistência imediata e a atomicidade. Uma transferência deve confirmar todas as alterações ou nenhuma delas. O schema-on-write ajuda a controlar a estrutura e as regras dos dados, enquanto as transações ACID garantem a integridade. O histórico também pode ser organizado em tabelas relacionadas, facilitando auditoria.

2.2 Cenário B — Placa de Captura de Dados IoT e Telemetria de Veículos

São 50.000 veículos enviando dados a cada 2 segundos, resultando em aproximadamente 25.000 requisições por segundo. Os sensores também podem enviar estruturas diferentes dependendo do veículo e do fabricante.

Item de Avaliação	Descrição dos Requisitos
Paradigma Escolhido	NoSQL — Colunar (wide-column). É adequado para grandes volumes de dados distribuídos e alta taxa de escrita.
SGBD Recomendado	Apache Cassandra. É um banco distribuído projetado para alta disponibilidade, baixa latência e escalabilidade horizontal [8].
Justificativa Técnica	A carga exige principalmente scale-out, pois os dados podem ser distribuídos entre vários nós. O Cassandra trabalha com particionamento e permite adicionar nós conforme a demanda. O modelo flexível também atende à variação dos dados dos sensores. Nesse caso, BASE e consistência eventual podem ser aceitos porque a prioridade é manter a ingestão disponível mesmo durante falhas ou partições.

2.3 Cenário C — Catálogo de Produtos para E-commerce Global

O catálogo possui produtos muito diferentes entre si e novos tipos são adicionados diariamente. Além da flexibilidade, o sistema precisa apresentar baixa latência nas consultas.

Item de Avaliação	Descrição dos Requisitos
Paradigma Escolhido	NoSQL — Documentos. O modelo permite que diferentes produtos possuam diferentes conjuntos de atributos.
SGBD Recomendado	MongoDB. Utiliza documentos flexíveis e permite indexação e distribuição horizontal [6].

Item de Avaliação	Descrição dos Requisitos
Justificativa Técnica	Schema-on-read e documentos flexíveis evitam que o banco precise ser alterado sempre que surgir um novo tipo de produto. Uma roupa pode possuir tamanho, cor e tecido, enquanto um eletrônico pode possuir processador, RAM e voltagem. Os dados podem ser organizados conforme o padrão de consulta, reduzindo a necessidade de relacionamentos complexos e favorecendo baixa latência. O sistema também pode utilizar scale-out para atender ao crescimento global.

3. REFERÊNCIAS

[1] POSTGRESQL GLOBAL DEVELOPMENT GROUP. Transaction Processing. PostgreSQL Documentation. Disponível em: https://www.postgresql.org/docs/current/transactions.html. Acesso em: 6 out. 2026.

[2] BAKHTIARI, M. et al. BASE analysis of NoSQL database. Future Generation Computer Systems, v. 52, p. 13–21, 2015. DOI: 10.1016/j.future.2015.05.003.

[3] GILBERT, S.; LYNCH, N. Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. ACM SIGACT News, v. 33, n. 2, 2002. DOI: 10.1145/564585.564601.

[4] IBM. Schema on Read vs. Schema on Write. IBM Think. Disponível em: https://www.ibm.com/think/topics/schema-on-read-vs-schema-on-write. Acesso em: 6 out. 2026.

[5] AWS. Sharding with Amazon Relational Database Service. AWS Database Blog. Disponível em: https://aws.amazon.com/blogs/database/sharding-with-amazon-relational-database-service/. Acesso em: 6 out. 2026.

[6] MONGODB. Data Modeling. MongoDB Documentation. Disponível em: https://www.mongodb.com/docs/manual/core/data-modeling-introduction/. Acesso em: 6 out. 2026.

[7] REDIS. Use cases. Redis Documentation. Disponível em: https://redis.io/docs/latest/develop/use-cases/. Acesso em: 6 out. 2026.

[8] APACHE CASSANDRA. Overview. Apache Cassandra Documentation. Disponível em: https://cassandra.apache.org/doc/latest/cassandra/architecture/overview.html. Acesso em: 6 out. 2026.

[9] NEO4J. What is a graph database?. Neo4j Documentation. Disponível em: https://neo4j.com/docs/getting-started/graph-database/. Acesso em: 6 out. 2026.